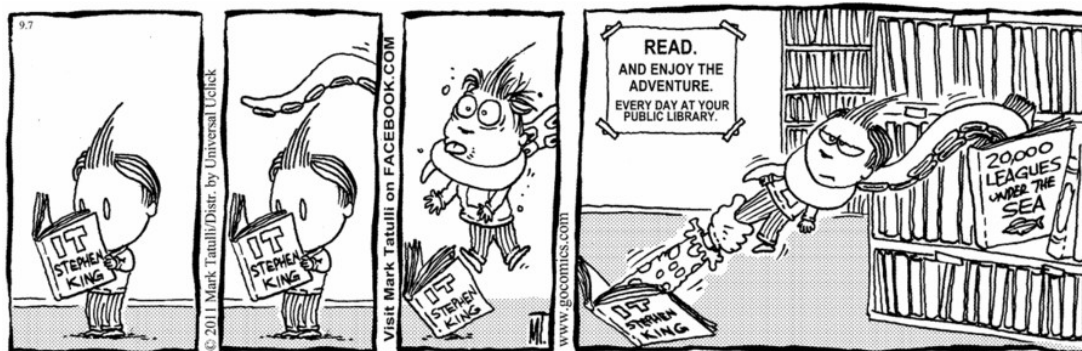


Cahier des charges

Réalisation d'un programme en C

« La bibliothèque »



Superviseurs :

Cauchie A.
(a.cauchie@indbg.be)

Embrechts L.
(l.embrechts@indbg.be)

Réalisé par :
[Nom et prénom de l'élève]

1. Définition du projet

1.1. Description de la problématique

Une bibliothèque communautaire fait appel à vous afin de créer un système informatique pour gérer son stock et la location de ses livres. L'objectif de ce projet est de mettre en place un logiciel qui réponde le plus possible aux attentes des commanditaires.

Une rencontre avec le président et les différents bénévoles a permis de déterminer ce qu'ils souhaiteraient mettre en place. Cette rencontre a été retranscrite par écrit et est disponibles dans les annexes de ce document.

1.2. Délivrables

1.2.1. Échéances

Délivrable	Échéance
Interfaces et cas d'utilisation	19 mars
Description des données	26 mars
Diagramme(s) d'action	2 avril
La structure générale du code (prototypes des fonctions, structures, etc.)	16 avril
L'implémentation des fonctions	14 mai
Le projet complet	19 mai

1.2.2. Consignes de remise

Le programme (code source, compilé et exécutable) et toutes les ressources nécessaires à son exécution ainsi que le cahier des charges (format PDF) doivent être remis dans un dossier nommé LangC_5TT_Nom_Prénom. Ce dossier sera déposé sur la plateforme « Classroom ». **Aucun retard ne sera accepté sauf cas de force majeure.** En cas de non remise à la date indiquée, une cote nulle sera attribuée.

1.2.3. Évaluation

Évaluation du cahier des charges

Le cahier des charges (ce document) doit être complété. La partie 1 est complète, à vous de compléter les autres.

La mise en page, l'orthographe et la qualité de la rédaction seront prises en compte dans l'évaluation.

Évaluation du programme

Le programme sera évalué en fonction des points suivants :

- le programme s'exécute correctement au niveau des fonctionnalités attendues,
- le code source est de qualité ou non

Des bonus seront attribués pour :

- l'allocation dynamique
- ou des fonctionnalités supplémentaires non mentionnées dans le cahier des charges

Toutefois, ces bonus ne seront attribués que si la note du travail de base atteint les 80%.

Évaluation orale

L'évaluation orale consiste à présenter le projet oralement devant les superviseurs. La présentation se déroulera comme il suit :

1. Présentation du logiciel
2. Présentation du code source
3. Crash test : un autre élève teste (sans consigne du développeur) le logiciel, sa robustesse, sa prise en main
4. Questions / Réponses

La présentation du logiciel ainsi que la présentation du code source ne devra pas dépasser 10 minutes.

1.3. Technologies utilisées

1.3.1. Langage de programmation

Le programme doit être rédigé en langage C.

1.3.2.Librairies

Aucune librairie supplémentaire non vue a cours ne peut être utilisée sauf accord des superviseurs.

2. Analyse et conception

2.1. Interfaces et cas d'utilisation

Interfaces de début

1. Gestion des livres
 2. Gestion des membres
 3. Gestion des emprunts et des remises des livres
-

Interfaces pour le 1 (gestion des livres)

1. Ajouter un nouveau livre
 2. Modifier un livre
 3. Lister les livres
 4. Recherche dans la liste des livres
 5. Supprimer un livre
-

Interfaces pour le 1.1 (ajouter un nouveau livre) CAS 1 quand l'utilisateur entre un ISBN valide

Entrez le code ISBN à (10 chiffres) du livre :
> 2072915813

Interfaces pour le 1.1 (ajouter un nouveau livre) CAS 2 quand l'utilisateur entre un ISBN qui ne fait pas 10 chiffres + boucles tant qu'il n'entre pas un ISBN valide

Entrez le code ISBN à (10 chiffres) du livre :
> 2072915813
le code ISBN ne contient pas 10 chiffres

Entrez le code ISBN à (10 chiffres) du livre :

>

Interfaces pour le 1.1(ajouter un nouveau livre) CAS 1 quand l'utilisateur entre le titre du livre est valide

Entrez le code ISBN à (10 chiffres) du livre :

> 2072915813

Entrez le titre du livre :

> les enfants sont rois

Interfaces pour le 1.1(ajouter un nouveau livre) CAS 1 quand l'utilisateur entre un auteur de livre valide

Entrez le code ISBN à (10 chiffres) du livre :

> 2072915813

Entrez le titre du livre :

> les enfants sont rois

Entrez l'auteur du livre :

> Delphine de vigan

Interfaces pour le 1.1(ajouter un nouveau livre) CAS 1 quand l'utilisateur entre un éditeur de livre valide

Entrez le code ISBN à (10 chiffres) du livre :

> 2072915813

Entrez le titre du livre :

> les enfants sont rois

Entrez l'auteur du livre :

> Delphine de vigan

Entrez l'éditeur :

> Gallimard

Interfaces pour le 1.1(ajouter un nouveau livre) CAS 1 quand l'utilisateur entre l'année de parution valide

Entrez le code ISBN à (10 chiffres) du livre :
> 2072915813
Entrez le titre du livre :
> les enfants sont rois
Entrez l'auteur du livre :
> Delphine de vigan
Entrez l'éditeur :
> Gallimard
Entrez l'année de parution (tout en attaché) :
>2022

Interfaces pour le 1.1(ajouter un nouveau livre) CAS 2 quand l'utilisateur entre l'année de parution que n'existe pas encore comme 2025

Entrez le code ISBN à (10 chiffres) du livre :
> 2072915813
Entrez le titre du livre :
> les enfants sont rois
Entrez l'auteur du livre :
> Delphine de vigan
Entrez l'éditeur :
> Gallimard
Entrez l'année de parution (tout en attaché) :
>2022
l'année de parution est invalide

Interfaces pour le 1.2(Modifier un livre)

Entrez le nom du livre que vous voulez modifier :

> Les enfants sont rois

Interfaces pour le 1.2(Modifier un livre)

Entrez le nom du livre que vous voulez modifier :

> Les enfants sont rois

voici la liste des livres que vous avez recherchés :

Num	ISBN	Auteur	Editeur	Titre	Année de parution	Disponibilité
1	2072915813	Delphine de vigan	Galimard	les enfants sont rois	2021	oui
2	2092570854	Thierry Courtin	Galimard	Les enfants sont rois	2020	non

Interfaces pour le 1.2(Modifier un livre) CAS 1 il veut modifier et on passe à la prochaine interface OU CAS 2 il ne veut pas et il est rediriger au menu.

Entrez le nom du livre que vous voulez modifier :

> Les enfants sont rois

voici la liste des livres que vous avez recherchés :

2072915813	Delphine de vigan	Galimard	les enfants sont rois	2021	oui
2092570854	Thierry Courtin	Galimard	Les enfants sont rois	2020	non

2451959467 Mathieu Niels JF Mathieu et ses amis 2007
oui

Etes-vous sûr de vouloir modifier ce livre :

Entrez o = oui ou n = non :

>o

Interfaces pour le 1.2(Modifier un livre)

Entrez le nom du livre que vous voulez modifier :

> Les enfants sont rois

voici la liste des livres que vous avez recherchés :

2072915813 Delphine de vigan Galimard les enfants sont rois 2021
oui

2092570854 Thierry Courtin Galimard Les enfants sont rois 2020
non

2451959467 Mathieu Niels JF Mathieu et ses amis 2007
oui

Etes-vous sûr de vouloir modifier ce livre :

Entrez o = oui ou n = non :

>o

Entrez le nom de ce que vous voulez modifier (C = CODE, A = AUTEUR, E = EDITEUR, P = PARUTION, T = TITRE) :

>A

Interfaces pour le 1.2(Modifier un livre)

Entrez le nom du livre que vous voulez modifier :

> Les enfants sont rois

voici la liste des livres que vous avez recherchés :

2072915813 Delphine de vigan Galimard les enfants sont rois 2021
oui

2092570854 Thierry Courtin Galimard Les enfants sont rois 2020
non

2451959467 Mathieu Niels JF Mathieu et ses amis 2007
oui

Etes-vous sûr de vouloir modifier ce livre :

Entrez o = oui ou n = non :

>o

Entrez le nom de ce que vous voulez modifier (C = CODE, A = AUTEUR, E = EDITEUR, P = PARUTION, T = TITRE) :

>A

Entrez-le nouveau/nouvelle auteur du livre :

> Thierry Courtin

Interfaces pour le 1.2(Modifier un livre) CAS 1(La motif est prise en compte)

Entrez le nom du livre que vous voulez modifier :

> Les enfants sont rois

voici la liste des livres que vous avez recherchés :

2072915813 Delphine de vigan Galimard les enfants sont rois 2021
libre

2092570854 Thierry Courtin Galimard Les enfants sont rois 2020
libre

2451959467 Mathieu Niels JF Mathieu et ses amis 2007
libre

Etes-vous sûr de vouloir modifier ce livre :

Entrez o = oui ou n = non :

>o

Entrez le nom de ce que vous voulez modifier (C = CODE, A = AUTEUR, E = EDITEUR, P = PARUTION, T = TITRE) :

>A

Entrez-le nouveau/nouvelle auteur du livre :

> Thierry Courtin

La modification à bien été prise en compte

Interface1.3(listerleslivres)

2072915813 Delphine de vigan Galimard les enfants sont rois 2021
libre

2092570854 Thierry Courtin Galimard Les enfants sont rois 2020
libre

2451959467	Mathieu Niels	JF	Mathieu et ses amis	2007	
libre					

Interfaces pour le 1.4(recherche dans la liste des livres)

Entrez le nom du livre que vous voulez rechercher :

> Les enfants sont rois

voici la liste des livres que vous avez recherchés :

2072915813	Delphine de vigan	Galimard	les enfants sont rois	2021	
libre					

2092570854	Thierry Courtin	Galimard	Les enfants sont rois	2020	
libre					

2451959467	Mathieu Niels	JF	Mathieu et ses amis	2007	
libre					

1.Ajouter un nouveau livre

2.Modifier un livre

3.Lister les livres

4.Recherche dans la liste des livres

5.Supprimer un livre

Interfaces pour le 1.5(supprimer un livre) CAS 1(s'il entre oui on le supprime et on propose de retourner au menu principal, s'il entre non on recommence le programme)

Entrez le nom du livre que vous voulez supprimer :

> Les enfants sont rois

voici la liste des livres que vous avez recherchés :

2072915813 Delphine de vigan Galimard les enfants sont rois 2021
libre

2092570854 Thierry Courtin Galimard Les enfants sont rois 2020
libre

2451959467 Mathieu Niels JF Mathieu et ses amis 2007
libre

Etes-vous sûr de vouloir supprimer ce livre :

Entrez o = oui ou n = non :

>o

Interfaces pour le 2(gestion des membres)

1.enregistrer un nouveau membre

2.modifier les données d'un membre

3.supprimer un membre

Interfaces pour le 2.1(enregistrer un nouveau membre) CAS 1 s'il entre un ID qui existe déjà on redemande tant qu'il n'entre pas un ID valide CAS 2 s'il entre oui on lui indique que le membre est bien ajouté et on l'ajoute OU s'il entre non on le redirige vers le menu du début

Entrez l'ID du nouveau membre :

>1

Entrez le prénom du nouveau membre :

>Mathieu

Entrez le nom du nouveau membre :

>Niels

Entrez l'adresse du nouveau membre (rue, numéro, code postal, ville/village) :

>rue du petit quartier,11,5575, Malvoisin

Etes-vous sûr de vouloir ajouter ce membre :

Entrez o = oui ou n = non :

>o

Interfaces pour le 2.2(modifier les données d'un membre) CAS 1 s'il entre un ID qui existe déjà on redemande tant qu'il n'entre pas un ID valide CAS 2 s'il entre non on le redirige vers le menu.

Entrez L'ID du membre que vous voulez modifier :

>1

| Mathieu | Niels | rue du petit quartier,11,5575, Malvoisin | 24/03/2023 |

| Thomas | Niels | rue des 4 seigneurs,11,5575, Malvoisin | 23/03/2023 |

Etes-vous sûr de vouloir modifier ce membre :

Entrez o = oui ou n = non :

>o

Entrez le nom de ce que vous voulez modifier (p = PRENOM, n = NOM, a=ADRESSE) :

>p

Entrez le nouveau prénom du membre :

> pavel

La modification à bien été prise en compte

Interfaces pour le 2.3(supprimer un nouveau membre) CAS (s'il entre oui on le supprime, s'il entre non on le redirige vers le menu principal

Entrez L'ID du membre que vous voulez supprimer :

>1

| Mathieu | Niels | rue du petit quartier,11,5575, Malvoisin | 24/03/2023 |

| Thomas | Niels | rue des 4 seigneurs,11,5575, Malvoisin | 23/03/2023 |

Etes-vous sûr de vouloir supprimer ce membre :

Entrez o = oui ou n = non :

>o

Le membre à bien été supprimer

Interfaces pour le 2.3(lister les membres) CAS 1 s'il n'y a rien dans le fichier on indique une erreur OU si le fichier est rempli on affiche.

| Mathieu | Niels | rue du petit quartier,11,5575, Malvoisin | 24/03/2023 |

| Thomas | Niels | rue des 4 seigneurs,11,5575, Malvoisin | 23/03/2023 |

Interfaces pour le 3(gestion des emprunts et des remises des livres)

1.Emprunter des livres

- 2.Remise d'un livre
 - 3.Payer une amende
-

Interfaces pour le 3.1(emprunter des livres) CAS 1(si le membre existe on continue, si le membre n'existe pas on retourne au menu principal pour l'ajouter)

Entrez l'ID du membre qui veut emprunter un ouvrage :
>1

CAS 2 (si le membre à dépasser son nombre de livre qu'il peut emprunter, on le redirige au menu principal)

Entrez l'ID du membre qui veut emprunter un ouvrage :
>1

Vous avez dépassé le nombre de livre que vous pouvez emprunterait

- 1.emprunter des livres
 - 2.remise d'un livre
-

CAS 3 (si le membre à des amendes à payer on lui demande si il veut payer maintenant) (s'il le paye de suite on l'envoie vers la fonction payer une amende) (s'il ne paye pas tout de suite on le redirige vers le menu)

Entrez l'ID du membre qui veut emprunter un ouvrage :
>1

Vous avez une amende de 15€ à payer
souhaitez-vous payer tout de suite ?

Entrez o = oui ou n = non :
> o

Interfaces pour le 3.1(emprunter des livres) CAS (s'il entre oui on l'emprunt et on lui affiche le prix à payer et le renvoie au menu OU s'il entre non on le redirige vers le menu principal)

Entrez l'ID du membre qui veut emprunter un ouvrage :
>1

Etes-vous sûr de vouloir emprunter ce livre :
Entrez o = oui ou n = non :
>o

Interfaces pour le 3.2(remise d'un livre) CAS 1(s'il entre un livre pas emprunter on lui fait une boucle pour lui demander tant qu'il ne mets pas un livre emprunter) CAS 2 (si il a une amende à payer et s'il veut la payer maintenant on le renvoie vers la fonction payer amende OU si il ne veut pas la payer maintenant on le redirige vers le menu)

Entrez le code ISBN a (10 chiffres) du livre :
>2266111655

Interfaces pour le 3.2(remise d'un livre) CAS (s'il entre oui on le rend et on le redirige vers le menu OU il entre non et on le redirige vers le menu)

Entrez le code ISBN a (10 chiffres) du livre :
>2266111655
Etes-vous sûr de vouloir rendre ce livre :
Entrez o = oui ou n = non :
>o

Interfaces pour le 3.2 (payer une amende) CAS (si le membre existe on continue, si le membre n'existe pas on retourne au menu principal pour l'ajouter)

Entrez l'ID du membre qui a une amende à payer :

>1

Interfaces pour le 3.2 (payer une amende) CAS (s'il a une amende à payer on lui affiche le montant et on lui demande s'il veut la payer maintenant, si oui il l'a payé OU si non on le redirige vers le menu principal)

Entrez l'ID du membre qui a une amende à payer :

>1

Entrez le code ISBN a (10 chiffres) du livre :

>2266111655

2.2. Description des données

Description des données

Nom du fichier :

ouvrages.dat

Description des données contenue :

il contient les informations des livres.

Les informations des livres sont triés dans l'ordre ci-dessous :

ISBN|le titre du livre|l'auteur|l'année de parution|éditeur

Son type :

c'est un fichiers DAT

structure des données contenues :

Propriétés	Type	Description
ISBN	Tableau de char [11]	Code du livre
Titre	Tableau de char [100]	Le titre du livre
Auteur	Tableau de char [100]	Auteur du livre
Année de parution	int	Année de sortie du livre
Éditeur	Tableau de char [50]	Éditeur du livre

-

Nom du fichier :

membres.dat

Description des données contenue :

il contient les informations des membres.

Les informations des livres sont triés dans l'ordre ci-dessous :

ID | Nom | Prénom | Adresse |Date_Adhésion

Son type :

c'est un fichiers DAT

structure des données contenues :

Propriétés	Type	Description
ID	Tableau de char [11]	Identifiant du membre
Nom	Tableau de char [100]	Nom du membre
Prénom	Tableau de char [100]	Prénom du membre
Adresse	Tableau de char [200]	Adresse du membre
Date_Adhésion	int	Date d'adhésion a la bibliothèque en format AAAAMMJJ

Nom du fichier :

emprunts.dat

Description des données contenue :

il contient les informations des emprunts des livres.

Les informations des livres sont triés dans l'ordre ci-dessous :

ID | ISBN | Date_Emprunt

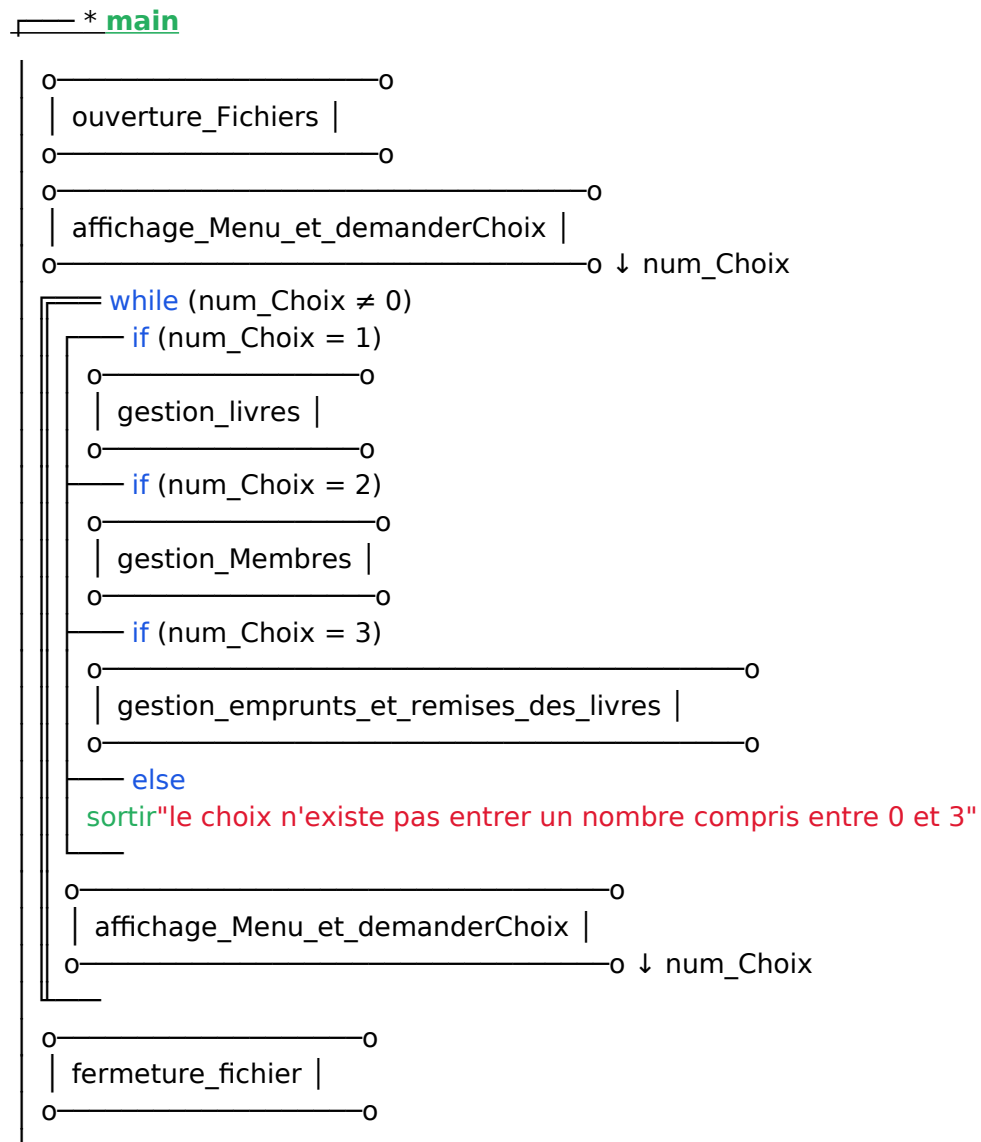
Son type :

c'est un fichiers DAT

structure des données contenues :

Propriétés	Type	Description
ID	Tableau de char [11]	Identifiant du membre
ISBN	Tableau de char [11]	Code du livre
Date_Emprunt	int	Date du jour de l'emprunt du livre dans le format AAAA-MM-JJ

2.3. Diagramme(s) d'action



3. Implémentation

3.1 Header.h

```
#include <stdio.h>
#include <stdbool.h>
#include <stdlib.h>
#include <ctype.h>
#include <string.h>
#include <time.h>
#pragma warning(disable : 4996)
#include "color.h"
#include "date_adhesion.h"

#define TAILLE_ISBN_OUVRAGE 11
#define TAILLE_TITRE_OUVRAGE 100
#define TAILLE_AUTEUR_OUVRAGE 100
#define TAILLE_EDITEUR_OUVRAGE 50
#define TAILLE_ID_MEMBRE 11
#define TAILLE_NOM_MEMBRE 100
#define TAILLE_PRENOM_MEMBRE 100
#define TAILLE_ADRESSE_MEMBRE 200
#define TAILLE_STATUT_OUVRAGE 15

#define COLOR_BOLD "\33[1m"
#define COLOR_OFF "\33[m"

void ouverture_Fichiers();
int affichage_Menu_et_demanderChoix(void);
void gestion_livres(void);
void gestion_membres(void);
void gestion_emprunts_et_remises_des_livres(void);
void ajouter_nouveau_livre(void);
void modifier_livre(void);
void lister_livre(void);
void recherche_livre(void);
void supprimer_livre(void);
void enregistrer_new_membre(void);
void modifier_donnee_membre(void);
void supprimer_membre(void);
int affichage_gestion_livres_demande_choix(void);
int affichage_gestion_membres_demande_choix(void);
int affichage_gestion_emprunts_remises_demande_choix(void);
void emprunts_ouvrages(void);
void remises_ouvrages(void);
```

```

void payer_amende(void);
void afficher_liste_membres(void);
void viderTampon(void);
int obtenir_annee(void);
struct Ouvrage {
    char isbn[TAILLE_ISBN_OUVRAGE];
    char titre[TAILLE_TITRE_OUVRAGE];
    char auteur[TAILLE_AUTEUR_OUVRAGE];
    int annee_parution;
    char editeur[TAILLE_EDITEUR_OUVRAGE];
};
typedef struct Ouvrage ouvrage;
struct Membre {
    int id_membre;
    char prenom[TAILLE_PRENOM_MEMBRE];
    char nom[TAILLE_NOM_MEMBRE];
    char adresse[TAILLE_ADRESSE_MEMBRE];
    int date_adhesion;
};
typedef struct Membre membre;
struct Emprunt {
    int id_membre;
    char isbn[TAILLE_ISBN_OUVRAGE];
    int date_emprunt;
};
typedef struct Emprunt emprunt;

void viderTampon(void) {
    int c;

    do {
        c = getchar();
    } while (c != '\n' && c != EOF);
}

int obtenir_annee(void) {
    time_t now;
    struct tm current_time;
    time(&now);
    current_time = *localtime(&now);

    return current_time.tm_year + 1900;
}

```


3.2 color.h

```
#include <stdio.h>

void red(void);
void cyan(void);
void reset(void);
void green(void);
void red(void) {
    printf("\033[1;31m");
}
void reset(void) {
    printf("\033[0m");
}
void cyan(void) {
    printf("\033[1;36m");
}
void green(void) {
    printf("\033[0;32m");
}
```

3.3 date_adhesion.h

```
#pragma once
#pragma warning(disable : 4996) // _CRT_SECURE_NO_WARNINGS
#include <time.h>
#include <stdint.h>

void extraireDepuisDateYYYYMMDD(int date, int* year, int* month, int* day);
int calculerNbJours(int dateStartYYYYMMDD, int dateEndYYYYMMDD);
int obtenirDateYYYYMMDDDuJour(void);
/*
* Entrées :
*   - <dateStartYYYYMMDD> : un entier représentant une date au format YYYYMMDD
*   - <dateEndYYYYMMDD> : un entier représentant une date au format YYYYMMDD
* Sorties :
*   - un entier représentant le nombre de jours entre les deux dates
*/
int calculerNbJours(int dateStartYYYYMMDD, int dateEndYYYYMMDD) {
    time_t now;
    struct tm date1;
    struct tm date2;
    double seconds;
    int extractedDay;
    int extractedMonth;
    int extractedYear;

    time(&now);

    date1 = *localtime(&now);
    date2 = *localtime(&now);
```

```

        extraireDepuisDateYYYYMMDD(dateStartYYYYMMDD, &extractedYear, &extractedMonth,
&extractedDay);
        date1.tm_hour = 0;
        date1.tm_min = 0;
        date1.tm_sec = 0;
        date1.tm_mon = extractedMonth - 1;
        date1.tm_mday = extractedDay;
        date1.tm_year = extractedYear - 1900;

        extraireDepuisDateYYYYMMDD(dateEndYYYYMMDD, &extractedYear, &extractedMonth,
&extractedDay);
        date2.tm_hour = 0;
        date2.tm_min = 0;
        date2.tm_sec = 0;
        date2.tm_mon = extractedMonth - 1;
        date2.tm_mday = extractedDay;
        date2.tm_year = extractedYear - 1900;

        seconds = difftime(mktime(&date2), mktime(&date1));

        return seconds / 86400;
}

/*
* Entrées : aucune
* Sortie : un entier représentant la date du jour au format YYYYMMDD
*/
int obtenirDateYYYYMMDDDuJour(void) {
    time_t now;
    struct tm* current_date;
    char buffer[9];
    int dateYYYYMMDD;

    time(&now);
    current_date = localtime(&now);
    strftime(buffer, 9, "%Y%m%d", current_date);
    dateYYYYMMDD = atoi(buffer);
    return dateYYYYMMDD;
}

/*
* Entrée: <date>, un entier représentant une date au format YYYYMMDD (ex. 20230405)
* Sorties :
*   - <year>      : l'année (ex. 2023)
*   - <month>     : le mois (ex. 4)
*   - <day>       : le jour (ex. 5)
*/
void extraireDepuisDateYYYYMMDD(int date, int* year, int* month, int* day) {
    *year = date / 10000;
    *month = (date - (*year * 10000)) / 100;
    *day = (date - (*year * 10000) - *month * 100);
}

```

3.4 source.c

```
#include "Header.h"
```

```
void main(void) {
    ouverture_Fichiers();
    int choix_menu_principal = affichage_Menu_et_demanderChoix();
    while (choix_menu_principal >= 1 && choix_menu_principal < 4)
    {
        switch (choix_menu_principal)
        {
            case 1:gestion_livres();
                break;
            case 2:gestion_membres();
                break;
            case 3:gestion_emprunts_et_remises_des_livres();
                break;
        }
        choix_menu_principal = affichage_Menu_et_demanderChoix();
    }
}
```

```
void ouverture_Fichiers() {
    FILE* pOuvrages;
    fopen_s(&pOuvrages, "ouvrages.dat", "rb+");
    if (pOuvrages == NULL) {
        fopen_s(&pOuvrages, "ouvrages.dat", "ab+");
        fclose(pOuvrages);
        fopen_s(&pOuvrages, "ouvrages.dat", "rb+");
    }
    fclose(pOuvrages);
    FILE* pMembre;
    fopen_s(&pMembre, "membre.dat", "rb+");
    if (pMembre == NULL) {
        fopen_s(&pMembre, "ouvrages.dat", "ab+");
        fclose(pOuvrages);
        fopen_s(&pMembre, "ouvrages.dat", "rb+");
    }
    fclose(pMembre);

    FILE* pEmprunt;
    fopen_s(&pEmprunt, "emprunt.dat", "rb+");
    if (pEmprunt == NULL) {
        fopen_s(&pEmprunt, "ouvrages.dat", "ab+");
        fclose(pOuvrages);
        fopen_s(&pEmprunt, "ouvrages.dat", "rb+");
    }
}
```

```

    }
    fclose(pEmprunt);

}
//affichage menu principal

int affichage_Menu_et_demanderChoix(void) {
    int choix_menu_principal;
    printf("-----\n");
    printf("1.Gestion des livres\n");
    printf("2.Gestion des membres\n");
    printf("3.Gestion des emprunts et des remises des livres\n");
    printf("4.Quitter\n");
    printf("-----\n");
    scanf_s("%d", &choix_menu_principal);
    viderTampon();
    system("cls");
    return choix_menu_principal;
}
//Gestion interfaces des livres

int affichage_gestion_livres_demande_choix(void) {
    int choix_gestion_livre;
    printf("-----\n");
    printf("1.Ajouter un nouveau livre\n");
    printf("2.Modifier un livre\n");
    printf("3.Lister les livres\n");
    printf("4.Recherche dans la liste des livres\n");
    printf("5.Supprimer un livre\n");
    printf("6.Quitter\n");
    printf("-----\n");
    scanf_s("%d", &choix_gestion_livre);
    viderTampon();
    system("cls");
    return choix_gestion_livre;
}

void gestion_livres(void) {
    int choix_gestion_livre = affichage_gestion_livres_demande_choix();
    while (choix_gestion_livre >= 1 && choix_gestion_livre < 6)
    {
        switch (choix_gestion_livre)
        {
            case 1:ajouter_nouveau_livre();
                break;
            case 2:modifier_livre();
                break;
            case 3:lister_livre();
                break;
            case 4:recherche_livre();
                break;
        }
    }
}

```

```

        case 5:supprimer_livre();
            break;
    }
    choix_gestion_livre = affichage_gestion_livres_demande_choix();
}
// Gestion des interfaces des membres

int affichage_gestion_membres_demande_choix(void) {
    int choix_gestion_membres;
    printf("-----\n");
    printf("1.Enregistrer un nouveau membre\n");
    printf("2.Modifier les donnees d'un membre\n");
    printf("3.afficher la liste des membres\n");
    printf("4.Supprimer un membre\n");
    printf("5.Quitter\n");
    printf("-----\n");
    scanf_s("%d", &choix_gestion_membres);
    viderTampon();
    system("cls");
    return choix_gestion_membres;
}

void gestion_membres(void) {
    int choix_gestion_membres = affichage_gestion_membres_demande_choix();
    while (choix_gestion_membres >= 1 && choix_gestion_membres < 5)
    {
        switch (choix_gestion_membres)
        {
            case 1:enregistrer_new_membre();
                break;
            case 2:modifier_donnee_membre();
                break;
            case 3:afficher_liste_membres();
                break;
            case 4:supprimer_membre();
            }
            choix_gestion_membres = affichage_gestion_membres_demande_choix();
        }
    }
}
//Gestion interfaces des membres

int affichage_gestion_emprunts_remises_demande_choix(void) {
    int choix_gestion_emprunts_et_remises;
    printf("-----\n");
    printf("1.Emprunter des livres\n");
    printf("2.Remise d'un livre\n");
    printf("3.Payer une amende\n");
    printf("4.Quitter\n");
    printf("-----\n");
    scanf_s("%d", &choix_gestion_emprunts_et_remises);

```

```

        viderTampon();
        system("cls");
        return choix_gestion_emprunts_et_remises;
    }

```

```

void gestion_emprunts_et_remises_des_livres(void) {
    int choix_gestion_emprunts_et_remises =
    affichage_gestion_emprunts_remises_demande_choix();
    while (choix_gestion_emprunts_et_remises >= 1 && choix_gestion_emprunts_et_remises
    < 4)
    {
        switch (choix_gestion_emprunts_et_remises)
        {
            case 1:emprunts_ouvrages();
                break;
            case 2:remises_ouvrages();
                break;
            case 3:payer_amende();
            }
        choix_gestion_emprunts_et_remises =
        affichage_gestion_emprunts_remises_demande_choix();
    }
}

```

```

void ajouter_nouveau_livre(void) {
    FILE* pOuvrages;
    ouvrage struct_ajout_ouvrage;
    ouvrage struct_ajout_ouvrage_bd;
    char validation_ajout_livre;
    int date_aujourd'hui = obtenir_annee();
    bool verif_nouveau_isbn = true;

```

```

    //code isbn
    do
    {
        verif_nouveau_isbn = true;
        fopen_s(&pOuvrages, "ouvrages.dat", "rb+");
        printf(COLOR_BOLD "Entrez le code ISBN a(10 chiffre) du livre :\n>"
        COLOR_OFF);
        cyan();
        scanf_s("%s", struct_ajout_ouvrage.isbn, TAILLE_ISBN_OUVRAGE);
        viderTampon();
        reset();

        while (strlen(struct_ajout_ouvrage.isbn) != 10)
        {
            red();
            printf("le code isbn ne contient pas 10 chiffres\n");

```

```

        reset();
        printf(COLOR_BOLD "Entrez le code ISBN a(10 chiffre) du livre :\n>"
COLOR_OFF);

        cyan();
        scanf_s("%s", struct_ajout_ouvrage.isbn, TAILLE_ISBN_OUVRAGE);
        viderTampon();
        reset();
    }
    while (fread_s(&struct_ajout_ouvrage_bd, sizeof(struct_ajout_ouvrage_bd),
sizeof(struct_ajout_ouvrage_bd), 1, pOuvrages) != 0)
    {
        if (strcmp(struct_ajout_ouvrage.isbn, struct_ajout_ouvrage_bd.isbn) == 0)
        {
            red();
            printf("le code isbn existe deja\n");
            reset();
            verf_nouveau_isbn = false;

        }
    }

    fclose(pOuvrages);
} while (!verif_nouveau_isbn);
fclose(pOuvrages);
fopen_s(&pOuvrages, "ouvrages.dat", "rb+");

//demande titre
printf(COLOR_BOLD "Entrez le titre du livre:\n>" COLOR_OFF);
cyan();
gets_s(struct_ajout_ouvrage.titre, TAILLE_TITRE_OUVRAGE);
reset();

//demande auteur
printf(COLOR_BOLD "Entrez l'auteur du livre:\n>" COLOR_OFF);
cyan();
gets_s(struct_ajout_ouvrage.auteur, TAILLE_AUTEUR_OUVRAGE);
reset();

//demande éditeur
printf(COLOR_BOLD "Entrez l'editeur du livre:\n>" COLOR_OFF);
cyan();
gets_s(struct_ajout_ouvrage.editeur, TAILLE_EDITEUR_OUVRAGE);
reset();

//demande année de parution
printf(COLOR_BOLD "Entrez l'annee de parution(tout en attache):\n>" COLOR_OFF);
cyan();
scanf_s("%d", &struct_ajout_ouvrage.annee_parution);
viderTampon();
reset();

```

```

while (struct_ajout_ouvrage.annee_parution > date_aujourd'hui)
{
    red();
    printf("l'annee de parution est invalide\n");
    reset();
    printf(COLOR_BOLD "Entrez l'annee de parution(tout en attache):\n>"
COLOR_OFF);
    cyan();
    scanf_s("%d", &struct_ajout_ouvrage.annee_parution);
    viderTampon();
    reset();

}
fclose(pOuvrages);
fopen_s(&pOuvrages, "ouvrages.dat", "ab+");
//validation
printf(COLOR_BOLD "Etes-vous sur de vouloir ajouter ce livre?\n" COLOR_OFF);
printf(COLOR_BOLD "Entrez o = oui ou n = non:\n>" COLOR_OFF);
cyan();
scanf_s("%c", &validation_ajout_livre, 1);
viderTampon();
reset();
if (validation_ajout_livre == 'o')
{
    fwrite(&struct_ajout_ouvrage, sizeof(struct_ajout_ouvrage), 1, pOuvrages);
    green();
    printf("le livre a bien ete emprunte\n");
    reset();
}

fclose(pOuvrages);
}

```

```

void modifier_livre(void) {
    FILE* pOuvrages;
    ouvrage struct_modifier_ouvrages;
    ouvrage struct_modifier_ouvrages_bd;
    ouvrage struct_nouvelle_modification_ouvrages;
    char validation_modification_livre;
    char choix_modification_livre;
    fopen_s(&pOuvrages, "ouvrages.dat", "rb+");
    printf(COLOR_BOLD "Entrez le titre du livre que vous voulez modifier :\n>"
COLOR_OFF);
    cyan();
    gets_s(struct_modifier_ouvrages.titre, TAILLE_TITRE_OUVRAGE);
    reset();
    while (fread_s(&struct_modifier_ouvrages_bd, sizeof(struct_modifier_ouvrages_bd),
sizeof(struct_modifier_ouvrages_bd), 1, pOuvrages) != 0 &&
strcmp(struct_modifier_ouvrages.titre, struct_modifier_ouvrages_bd.titre) != 0)
    {
    };
}

```



```

if (feof(pOuvrages))
{
    red();
    printf("le livre est introuvable\n");
    reset();
}
else
{
    printf("%11s|%39s|%30s|%7d|%25s\n", struct_modifier_ouvrages_bd.isbn,
struct_modifier_ouvrages_bd.titre, struct_modifier_ouvrages_bd.auteur,
struct_modifier_ouvrages_bd.annee_parution, struct_modifier_ouvrages_bd.editeur);
}
    printf(COLOR_BOLD "Etes-vous sur de vouloir modifier ce livre?\n" COLOR_OFF);
    printf(COLOR_BOLD "Entrez o = oui ou n = non:\n>" COLOR_OFF);
    cyan();
    scanf_s("%c", &validation_modification_livre, 1);
    viderTampon();
    reset();
    if (validation_modification_livre == 'o')
    {
        printf(COLOR_BOLD "Entrez le nom de ce que vous voulez modifier(i = isbn, a =
AUTEUR, e = EDITEUR, p = PARUTION, t = TITRE) :\n>" COLOR_OFF);
        cyan();
        scanf_s("%c", &choix_modification_livre, 1);
        viderTampon();
        reset();
        switch (choix_modification_livre)
        {
            case 'i':
                printf(COLOR_BOLD "Entrez le nouveau code isbn du livre\n>"
COLOR_OFF);
                cyan();
                gets_s(struct_nouvelle_modification_ouvrages.isbn,
TAILLE_ISBN_OUVRAGE);
                reset();

                fseek(pOuvrages, -1 * (long)sizeof(struct_modifier_ouvrages_bd),
SEEK_CUR);

                strcpy_s(struct_modifier_ouvrages_bd.isbn, TAILLE_ISBN_OUVRAGE,
struct_nouvelle_modification_ouvrages.isbn);
                fwrite(&struct_modifier_ouvrages_bd, sizeof(struct_modifier_ouvrages_bd),
1, pOuvrages);

                printf("la modification a bine été prise en compte");

                break;
            case 'a':
                printf(COLOR_BOLD "Entrez le nouveau/nouvelle auteur du livre\n>"
COLOR_OFF);
                cyan();
                gets_s(struct_nouvelle_modification_ouvrages.auteur,
TAILLE_AUTEUR_OUVRAGE);

```

```

        reset();

        fseek(pOuvrages, -1 * (long)sizeof(struct_modifier_ouvrages_bd),
SEEK_CUR);
        strcpy_s(struct_modifier_ouvrages_bd.auteur,
TAILLE_AUTEUR_OUVRAGE, struct_nouvelle_modification_ouvrages.auteur);
        fwrite(&struct_modifier_ouvrages_bd, sizeof(struct_modifier_ouvrages_bd),
1, pOuvrages);
        printf("la modification a bine été prise en compte");

        break;
    case 'e':
        printf(COLOR_BOLD "Entrez le nouveau/nouvelle editeur du livre\n>"
COLOR_OFF);
        cyan();
        gets_s(struct_nouvelle_modification_ouvrages.editeur,
TAILLE_EDITEUR_OUVRAGE);
        reset();

        fseek(pOuvrages, -1 * (long)sizeof(struct_modifier_ouvrages_bd),
SEEK_CUR);
        strcpy_s(struct_modifier_ouvrages_bd.editeur,
TAILLE_EDITEUR_OUVRAGE, struct_nouvelle_modification_ouvrages.editeur);
        fwrite(&struct_modifier_ouvrages_bd, sizeof(struct_modifier_ouvrages_bd),
1, pOuvrages);
        printf("la modification a bine été prise en compte");

        break;
    case 'p':
        printf(COLOR_BOLD "Entrez la nouvelle année de parution du livre\n>"
COLOR_OFF);
        cyan();
        scanf_s("%d", &struct_nouvelle_modification_ouvrages.annee_parution);
        viderTampon();
        reset();

        fseek(pOuvrages, -1 * (long)sizeof(struct_modifier_ouvrages_bd),
SEEK_CUR);
        struct_modifier_ouvrages_bd.annee_parution =
struct_nouvelle_modification_ouvrages.annee_parution;
        fwrite(&struct_modifier_ouvrages_bd, sizeof(struct_modifier_ouvrages_bd),
1, pOuvrages);
        printf("la modification a bine été prise en compte");

        break;
    case 't':
        printf(COLOR_BOLD "Entrez le nouveau titre du livre\n>" COLOR_OFF);
        cyan();
        gets_s(struct_nouvelle_modification_ouvrages.titre,
TAILLE_TITRE_OUVRAGE);
        reset();

```

```

        fseek(pOuvrages, -1 * (long)sizeof(struct_modifier_ouvrages_bd),
SEEK_CUR);
        strcpy_s(struct_modifier_ouvrages_bd.titre, TAILLE_TITRE_OUVRAGE,
struct_nouvelle_modification_ouvrages.titre);
        fwrite(&struct_modifier_ouvrages_bd, sizeof(struct_modifier_ouvrages_bd),
1, pOuvrages);
        printf("la modification a bine été prise en compte");
        break;
    }
}
else {
    gestion_livres();
}

fclose(pOuvrages);
}

```

```

void lister_livre(void) {
    FILE* pOuvrages;
    FILE* pEmprunt;
    ouvrage struct_liste_ouvrages;
    emprunt struct_acces_emprunt_bd;
    char statut[TAILLE_STATUT_OUVRAGE];

    fopen_s(&pOuvrages, "ouvrages.dat", "rb+");
    fopen_s(&pEmprunt, "emprunt.dat", "rb+");

    while (fread_s(&struct_liste_ouvrages, sizeof(struct_liste_ouvrages),
sizeof(struct_liste_ouvrages), 1, pOuvrages) != 0)
    {
        if (strcmp(struct_liste_ouvrages.titre, "****") != 0)
        {
            while (fread_s(&struct_acces_emprunt_bd,
sizeof(struct_acces_emprunt_bd), sizeof(struct_acces_emprunt_bd), 1, pEmprunt) != 0 &&
strcmp(struct_acces_emprunt_bd.isbn, struct_liste_ouvrages.isbn) != 0)
            {
            };
            if (strcmp(struct_acces_emprunt_bd.isbn, struct_liste_ouvrages.isbn) == 0)
            {
                strcpy_s(statut, TAILLE_STATUT_OUVRAGE, "emprunte");
            }
            else
            {
                strcpy_s(statut, TAILLE_STATUT_OUVRAGE, "libre");
            }
        }
    }
}

```

```

                printf("\n%5s|%11s|%35s|%27s|%4d|%22s\n\n", statut,
struct_lister_ouvrages.isbn, struct_lister_ouvrages.titre, struct_lister_ouvrages.auteur,
struct_lister_ouvrages.annee_parution, struct_lister_ouvrages.editeur);
            }
            fclose(pEmprunt);
            fopen_s(&pEmprunt, "emprunt.dat", "rb+");
        }

        fclose(pOuvrages);
        fclose(pEmprunt);
    }

void recherche_livre(void) {
    FILE* pOuvrages;
    ouvrage struct_rechercher_ouvrages;
    ouvrage struct_rechercher_ouvrages_bd;
    fopen_s(&pOuvrages, "ouvrages.dat", "rb");
    printf(COLOR_BOLD "Entrez le titre du livre que vous voulez rechercher :\n>"
COLOR_OFF);
    cyan();
    gets_s(struct_rechercher_ouvrages.titre, TAILLE_TITRE_OUVRAGE);
    reset();

    while (fread_s(&struct_rechercher_ouvrages_bd, sizeof(struct_rechercher_ouvrages_bd),
sizeof(struct_rechercher_ouvrages_bd), 1, pOuvrages) != 0 &&
strcmp(struct_rechercher_ouvrages.titre, struct_rechercher_ouvrages_bd.titre) != 0)
    {
    };

    if (feof(pOuvrages))
    {
        red();
        printf("le livre est introuvable\n");
        reset();
    }
    else
    {
        printf("%11s|%39s|%30s|%7d|%25s\n", struct_rechercher_ouvrages_bd.isbn,
struct_rechercher_ouvrages_bd.titre, struct_rechercher_ouvrages_bd.auteur,
struct_rechercher_ouvrages_bd.annee_parution, struct_rechercher_ouvrages_bd.editeur);
    }
    fclose(pOuvrages);
}

void supprimer_livre(void) {
    FILE* pOuvrages;
    ouvrage struct_supprimer_ouvrages;
    ouvrage struct_supprimer_ouvrages_bd;
    char validation_supp_livre;

```

```

    fopen_s(&pOuvrages, "ouvrages.dat", "rb+");
    printf(COLOR_BOLD "Entrez le titre du livre que vous voulez supprimer :\n>"
COLOR_OFF);
    cyan();
    gets_s(struct_supprimer_ouvrages.titre, TAILLE_TITRE_OUVRAGE);
    reset();

    while (fread_s(&struct_supprimer_ouvrages_bd, sizeof(struct_supprimer_ouvrages_bd),
sizeof(struct_supprimer_ouvrages_bd), 1, pOuvrages) != 0 &&
strcmp(struct_supprimer_ouvrages.titre, struct_supprimer_ouvrages_bd.titre) != 0)
    {

    if (feof(pOuvrages))
    {
        red();
        printf("le livre est introuvable\n");
        reset();
    }
    else
    {
        printf("%11s|%39s|%30s|%7d|%25s\n", struct_supprimer_ouvrages_bd.isbn,
struct_supprimer_ouvrages_bd.titre, struct_supprimer_ouvrages_bd.auteur,
struct_supprimer_ouvrages_bd.annee_parution, struct_supprimer_ouvrages_bd.editeur);
    }
    printf(COLOR_BOLD "Etes-vous sur de vouloir supprimer ce livre?\n" COLOR_OFF);
    printf(COLOR_BOLD "Entrez o = oui ou n = non:\n>" COLOR_OFF);
    cyan();
    scanf_s("%c", &validation_supp_livre, 1);
    viderTampon();
    reset();
    if (validation_supp_livre == 'o')
    {
        fseek(pOuvrages, -1 * (long)sizeof(struct_supprimer_ouvrages_bd), SEEK_CUR);
        strcpy_s(struct_supprimer_ouvrages_bd.titre, TAILLE_TITRE_OUVRAGE,
"***");
        fwrite(&struct_supprimer_ouvrages_bd, sizeof(struct_supprimer_ouvrages_bd), 1,
pOuvrages);
        green();
        printf("le livre a bien ete supprimer\n");
        reset();
    }
    else {
        gestion_livres();
    }
    fclose(pOuvrages);
}

```

```

void enregistrer_new_membre(void) {

```

```

FILE* pMembres;
FILE* pOuvrages;
membre struct_new_membre;
membre struct_new_membre_bd;
char validation_ajout_membre;
bool verif_nouveau_id_membre = true;

do
{
    verif_nouveau_id_membre = true;
    fopen_s(&pMembres, "membre.dat", "ab+");
    fopen_s(&pOuvrages, "ouvrages.dat", "rb+");
    printf(COLOR_BOLD "Entrez l'id du nouveau membre:\n>" COLOR_OFF);
    cyan();
    scanf_s("%d", &struct_new_membre.id_membre);
    viderTampon();
    reset();

    while (fread_s(&struct_new_membre_bd, sizeof(struct_new_membre_bd),
sizeof(struct_new_membre_bd), 1, pMembres) != 0)
    {
        if (struct_new_membre.id_membre == struct_new_membre_bd.id_membre)
        {
            red();
            printf("l'id du membre existe deja\n");
            reset();
            verif_nouveau_id_membre = false;
        }
    }

    fclose(pMembres);
    fclose(pOuvrages);
} while (!verif_nouveau_id_membre);

fopen_s(&pMembres, "membre.dat", "ab+");
fopen_s(&pOuvrages, "ouvrages.dat", "rb+");

printf(COLOR_BOLD "Entrez le prenom du nouveau membre :\n>" COLOR_OFF);
cyan();
gets_s(struct_new_membre.prenom, TAILLE_PRENOM_MEMBRE);
reset();

printf(COLOR_BOLD "Entrez le nom du nouveau membre :\n>" COLOR_OFF);
cyan();
gets_s(struct_new_membre.nom, TAILLE_NOM_MEMBRE);
reset();

printf(COLOR_BOLD "Entrez l'adresse du nouveau membre(rue,numero,code
postal,ville/village):\n>" COLOR_OFF);

```

```

cyan();
gets_s(struct_new_membre.adresse, TAILLE_ADRESSE_MEMBRE);
reset();

printf(COLOR_BOLD "Etes-vous sur de vouloir ajouter ce membre?\n");
printf(COLOR_BOLD "Entrez o = oui ou n = non:\n>");
cyan();
scanf_s("%c", &validation_ajout_membre, 1);
viderTampon();
reset();

if (validation_ajout_membre == 'o')
{
    struct_new_membre.date_adhesion = obtenirDateYYYYMMDDDuJour();
    fwrite(&struct_new_membre, sizeof(struct_new_membre), 1, pMembres);
}
else {
    gestion_membres();
}

fclose(pMembres);
fclose(pOuvrages);
}

void modifier_donnee_membre(void) {
    FILE* pMembres;
    membre struct_modifier_membre;
    membre struct_modifier_membre_bd;
    membre struct_nouvelle_modification_membre;
    char validation_modification_membre;
    char choix_modification_membre;
    printf(COLOR_BOLD "Entrez l'id du membre que vous voulez modifier:\n>"
COLOR_OFF);
    cyan();
    scanf_s("%d", &struct_modifier_membre.id_membre);
    viderTampon();
    reset();

    fopen_s(&pMembres, "membre.dat", "rb+");

    while (fread_s(&struct_modifier_membre_bd, sizeof(struct_modifier_membre_bd),
sizeof(struct_modifier_membre_bd), 1, pMembres) != 0 && struct_modifier_membre.id_membre != struct_modifier_membre_bd.id_membre)
    {
    };

    if (feof(pMembres))
    {

```

```

        red();
        printf("le livre est introuvable\n");
        reset();
    }
    else
    {
        printf("%11d|%22s|%22s|%50s|%8d\n", struct_modifier_membre_bd.id_membre,
struct_modifier_membre_bd.prenom, struct_modifier_membre_bd.nom,
struct_modifier_membre_bd.adresse, struct_modifier_membre_bd.date_adhesion);
    }
    printf(COLOR_BOLD "Etes-vous sur de vouloir modifier ce membre?\n" COLOR_OFF);
    printf(COLOR_BOLD "Entrez o = oui ou n = non:\n>" COLOR_OFF);
    cyan();
    scanf_s("%c", &validation_modification_membre, 1);
    viderTampon();
    reset();
    if (validation_modification_membre == 'o')
    {
        printf(COLOR_BOLD "Entrez le nom de ce que vous voulez modifier(p =
PRENOM, n = NOM, a = ADRESSE) :\n>");
        scanf_s("%c", &choix_modification_membre, 1);
        viderTampon();
        switch (choix_modification_membre)
        {
            case 'p':
                printf(COLOR_BOLD "Entrez le nouveau prenom du membre\n>"
COLOR_OFF);
                cyan();
                gets_s(struct_nouvelle_modification_membre.prenom,
TAILLE_ISBN_OUVRAGE);
                reset();

                fseek(pMembres, -1 * (long)sizeof(struct_modifier_membre_bd),
SEEK_CUR);
                strcpy_s(struct_modifier_membre_bd.prenom,
TAILLE_PRENOM_MEMBRE, struct_nouvelle_modification_membre.prenom);
                fwrite(&struct_modifier_membre_bd, sizeof(struct_modifier_membre_bd),
1, pMembres);
                printf("La modification à bien été prise en compte");

                break;
            case 'n':
                printf(COLOR_BOLD "Entrez le nouveau nom du membre\n>"
COLOR_OFF);
                cyan();
                gets_s(struct_nouvelle_modification_membre.nom,
TAILLE_AUTEUR_OUVRAGE);
                reset();

                fseek(pMembres, -1 * (long)sizeof(struct_modifier_membre_bd),
SEEK_CUR);
                strcpy_s(struct_modifier_membre_bd.nom, TAILLE_NOM_MEMBRE,

```



```

struct_nouvelle_modification_membre.nom);
        fwrite(&struct_modifier_membre_bd, sizeof(struct_modifier_membre_bd),
1, pMembres);
        printf("La modification à bien été prise en compte");

        break;
    case 'a':
        printf(COLOR_BOLD "Entrez la nouvelle adresse du membre\n>"
COLOR_OFF);
        cyan();
        gets_s(struct_nouvelle_modification_membre.adresse,
TAILLE_EDITEUR_OUVRAGE);
        reset();

        fseek(pMembres, -1 * (long)sizeof(struct_modifier_membre_bd),
SEEK_CUR);
        strcpy_s(struct_modifier_membre_bd.adresse,
TAILLE_ADRESSE_MEMBRE, struct_nouvelle_modification_membre.adresse);
        fwrite(&struct_modifier_membre_bd, sizeof(struct_modifier_membre_bd),
1, pMembres);
        printf("La modification à bien été prise en compte");

        break;
    }
}
else {
    gestion_membres();
}
fclose(pMembres);
}

```

```

void supprimer_membre(void) {
    FILE* pMembres;
    membre struct_supprimer_membre;
    membre struct_supprimer_membre_bd;
    char validation_supp_membre;
    fopen_s(&pMembres, "membre.dat", "rb+");

    printf(COLOR_BOLD "Entrez l'id du membre que vous voulez modifier:\n>"
COLOR_OFF);
    cyan();
    scanf_s("%d", &struct_supprimer_membre.id_membre);
    viderTampon();
    reset();

    while (fread_s(&struct_supprimer_membre_bd, sizeof(struct_supprimer_membre_bd),
sizeof(struct_supprimer_membre_bd), 1, pMembres) != 0 &&
struct_supprimer_membre.id_membre != struct_supprimer_membre_bd.id_membre)
    {
    };
}

```

```

    if (feof(pMembres))
    {
        red();
        printf("le membre est introuvable\n");
        reset();
    }
    else
    {
        printf("%11d|%22s|%22s|%50s|%8d\n\n",
struct_supprimer_membre_bd.id_membre, struct_supprimer_membre_bd.prenom,
struct_supprimer_membre_bd.nom, struct_supprimer_membre_bd.adresse,
struct_supprimer_membre_bd.date_adhesion);

        printf(COLOR_BOLD "Etes-vous sur de vouloir supprimer ce membre?\n"
COLOR_OFF);
        printf(COLOR_BOLD "Entrez o = oui ou n = non:\n>" COLOR_OFF);
        cyan();
        scanf_s("%c", &validation_supp_membre, 1);
        viderTampon();
        reset();
        if (validation_supp_membre == 'o')
        {
            fseek(pMembres, -1 * (long)sizeof(struct_supprimer_membre_bd),
SEEK_CUR);

            struct_supprimer_membre_bd.id_membre = 0;
            fwrite(&struct_supprimer_membre_bd,
sizeof(struct_supprimer_membre_bd), 1, pMembres);
            green();
            printf("le livre a bien ete supprimer\n");
            reset();
        }
        else {
            gestion_livres();
        }
    }
    fclose(pMembres);
}

```

```

void afficher_liste_membres(void) {
    FILE* pMembres;
    membre struct_lister_membres;
    fopen_s(&pMembres, "membre.dat", "rb+");

    // Vérifier si le fichier est vide
    fseek(pMembres, 0, SEEK_END); // Se déplacer à la fin du fichier
    if (ftell(pMembres) == 0) {
        red();
        printf("Le fichier est vide.\n");
    }
}

```

```

        reset();
        fclose(pMembres); // Fermer le fichier
        return;
    }
    fseek(pMembres, 0, SEEK_SET); // Se déplacer au début du fichier

    while (fread_s(&struct_lister_membres, sizeof(struct_lister_membres),
sizeof(struct_lister_membres), 1, pMembres) != 0)
    {

        printf("%11d|%22s|%22s|%50s|%8d\n\n", struct_lister_membres.id_membre,
struct_lister_membres.prenom, struct_lister_membres.nom, struct_lister_membres.adresse,
struct_lister_membres.date_adhesion);

    }

    fclose(pMembres);
}

```

```

void emprunts_ouvrages(void) {
    char validation_quitter_emprunt;

    FILE* pOuvrages;
    FILE* pMembres;
    FILE* pEmprunt;
    ouvrage struct_ajout_ouvrage;
    ouvrage struct_ajout_ouvrage_bd;
    membre struct_acces_membre;
    emprunt struct_emprunt_ouvrages;
    emprunt struct_emprunt_ouvrages_bd;
    char validation_payer_amende;
    char validation_emprunt_ouvrages;
    char validation_deja_emprunte;

    int nbEmprunts = 1;
    float montant_calcul_emprunt = 0;
    int ecart_date_emprunt;
    int date_auj = obtenirDateYYYYMMDDDuJour();
    bool decideDePayer = true;
    bool verif_nouveau_isbn = true;

    fopen_s(&pMembres, "membre.dat", "rb+");
    fopen_s(&pOuvrages, "ouvrages.dat", "rb+");
    fopen_s(&pEmprunt, "emprunt.dat", "rb+");
    printf(COLOR_BOLD "Entrez l'id du membre qui veux emprunter un ouvrage:\n>"
COLOR_OFF);
    cyan();
    scanf_s("%d", &struct_emprunt_ouvrages.id_membre);
    viderTampon();
    reset();
    do

```

```

{
    while (fread_s(&struct_acces_membre, sizeof(struct_acces_membre),
sizeof(struct_acces_membre), 1, pMembres) != 0 && struct_emprunt_ouvrages.id_membre !=
struct_acces_membre.id_membre)
    {
    };
    if (feof(pMembres))
    {
        printf(COLOR_BOLD "l'id du membre entre n'est pas valide\n"
COLOR_OFF);
        printf(COLOR_BOLD "Entrez l'id du membre qui veux emprunter un
ouvrage:\n>" COLOR_OFF);
        cyan();
        scanf_s("%d", &struct_emprunt_ouvrages.id_membre);
        viderTampon();
        reset();
    }
    fclose(pMembres);
    fopen_s(&pMembres, "membre.dat", "rb+");
} while (struct_emprunt_ouvrages.id_membre != struct_acces_membre.id_membre);

printf(COLOR_BOLD "Entrez le code ISBN a(10 chiffre) du livre :\n>" COLOR_OFF);
cyan();
gets_s(struct_emprunt_ouvrages.isbn, TAILLE_ISBN_OUVRAGE);
reset();

while (strlen(struct_emprunt_ouvrages.isbn) != 10)
{
    red();
    printf("le code isbn ne contient pas 10 chiffres\n");
    reset();
    printf(COLOR_BOLD "Entrez le code ISBN a(10 chiffre) du livre :\n>"
COLOR_OFF);
    cyan();
    gets_s(struct_emprunt_ouvrages.isbn, TAILLE_ISBN_OUVRAGE);
    reset();
}

while (fread_s(&struct_emprunt_ouvrages_bd, sizeof(struct_emprunt_ouvrages_bd),
sizeof(struct_emprunt_ouvrages_bd), 1, pEmprunt) != 0 && strcmp(struct_emprunt_ouvrages.isbn,
struct_emprunt_ouvrages_bd.isbn) != 0)
{
};
if (!feof(pEmprunt))
{
    red();
    printf("le livre est deja emprunte\n");
    reset();
}

```

```

    }
    else {

        fclose(pEmprunt);
        fopen_s(&pEmprunt, "emprunt.dat", "ab+");

        while (fread_s(&struct_emprunt_ouvrages_bd,
sizeof(struct_emprunt_ouvrages_bd), sizeof(struct_emprunt_ouvrages_bd), 1, pEmprunt) != 0 &&
decideDePayer && strcmp(struct_emprunt_ouvrages_bd.isbn, "****") != 0)
        {
            ecart_date_emprunt =
calculerNbJours(struct_emprunt_ouvrages_bd.date_emprunt, date_auj);
            if (struct_emprunt_ouvrages.id_membre ==
struct_emprunt_ouvrages_bd.id_membre)
            {
                nbEmprunts++;
                if (ecart_date_emprunt > 14)
                {
                    printf(COLOR_BOLD "le membre a emprunte un livre
depuis trop longtemps\n"COLOR_OFF);
                    printf(COLOR_BOLD "il a donc une amende a payer\n"
COLOR_OFF);
                    printf(COLOR_BOLD "voulez-vous payer l'amende
maintenant?:\n>" COLOR_OFF);
                    printf(COLOR_BOLD "Entrez o = oui ou n = non:\n>");
                    cyan();
                    scanf_s("%c", &validation_payer_amende, 1);
                    reset();
                    viderTampon();
                    if (validation_payer_amende == 'o')
                    {
                        payer_amende();
                    }
                    else
                    {
                        decideDePayer = false;
                    }
                }
            }

        }

    }

    }
    if (nbEmprunts > 10)
    {
        red();
        printf("le membre à deja emprunte trop de livre\n");
        printf("");
    }
    else

```

```

    {

        printf(COLOR_BOLD "Etes-vous sur de vouloir emprunter ce livre?\n"
COLOR_OFF);

        printf(COLOR_BOLD"Entrez o = oui ou n = non:\n>" COLOR_OFF);
        cyan();
        scanf_s("%c", &validation_emprunt_ouvrages, 1);
        viderTampon();
        reset();
        if (validation_emprunt_ouvrages == 'o')
        {
            struct_emprunt_ouvrages.date_emprunt =
obtenirDateYYYYMMDDDuJour();
            fwrite(&struct_emprunt_ouvrages, sizeof(struct_emprunt_ouvrages),
1, pEmprunt);

            green();
            printf("le livre a bien ete emprunte\n");
            reset();
            montant_calcul_emprunt = 0.75 * nbEmprunts;
            printf(COLOR_BOLD "vous devez payer %.2f euros pour l'emprunt
de ce livre\n" COLOR_OFF, montant_calcul_emprunt);
        }
    }

}

fclose(pMembres);
fclose(pOuvrages);
fclose(pEmprunt);
}

```

```

void remises_ouvrages(void) {
    FILE* pMembres;
    FILE* pEmprunt;
    emprunt struct_emprunt_ouvrages;
    emprunt struct_emprunt_ouvrages_bd;
    char validation_pas_emprunte;
    char validation_payer_amende;
    char validation_remises_ouvrages;
    int ecart_date_emprunt;
    bool decideDePayer = true;
    int date_auj = obtenirDateYYYYMMDDDuJour;
    fopen_s(&pMembres, "membre.dat", "rb+");
    fopen_s(&pEmprunt, "emprunt.dat", "rb+");
    printf(COLOR_BOLD "Entrez le code ISBN a(10 chiffre) du livre :\n>" COLOR_OFF);
    cyan();
    gets_s(struct_emprunt_ouvrages.isbn, TAILLE_ISBN_OUVRAGE);
    reset();
}

```



```

printf(COLOR_BOLD "Etes-vous sur de vouloir rendre ce livre?\n" COLOR_OFF);
printf(COLOR_BOLD "Entrez o = oui ou n = non:\n>" COLOR_OFF);
cyan();
scanf_s("%c", &validation_remises_ouvrages, 1);
viderTampon();
reset();
if (validation_remises_ouvrages == 'o')
{
    fseek(pEmprunt, -1 * (long)sizeof(struct_emprunt_ouvrages_bd),
SEEK_CUR);
    strcpy_s(struct_emprunt_ouvrages_bd.isbn, TAILLE_ISBN_OUVRAGE,
"***");
    fwrite(&struct_emprunt_ouvrages_bd, sizeof(struct_emprunt_ouvrages_bd),
1, pEmprunt);

    green();
    printf("le livre a bien ete rendu\n");
    reset();

}
else {
    gestion_livres();
}

}
fclose(pMembres);
fclose(pEmprunt);
}

```

```

void payer_amende(void) {
    FILE* pMembres;
    FILE* pEmprunt;
    membre struct_acces_membre;
    emprunt struct_emprunt_ouvrages;
    emprunt struct_emprunt_ouvrages_bd;
    char validation_payer_amende;
    int ecart_date_emprunt;
    int nb_calcul_amende;
    float montant_amende;
    int date_auj = obtenirDateYYYYMMDDDuJour();
    bool decideDePayer = true;
    fopen_s(&pMembres, "membre.dat", "rb+");
    fopen_s(&pEmprunt, "emprunt.dat", "rb+");
    printf(COLOR_BOLD "Entrez l'id du membre qui a une amende a payer:\n>"
COLOR_OFF);
    cyan();
    scanf_s("%d", &struct_emprunt_ouvrages.id_membre);
    viderTampon();
    reset();
}

```



```

do
{
    while (fread_s(&struct_acces_membre, sizeof(struct_acces_membre),
sizeof(struct_acces_membre), 1, pMembres) != 0 && struct_emprunt_ouvrages.id_membre !=
struct_acces_membre.id_membre)
    {
        };
        if (feof(pMembres))
        {
            red();
            printf("l'id du membre entre n'est pas valide\n");
            reset();
            printf(COLOR_BOLD "Entrez l'id du membre qui veut emprunter un
ouvrage:\n>" COLOR_OFF);
            cyan();
            scanf_s("%d", &struct_emprunt_ouvrages.id_membre);
            viderTampon();
            cyan();

        }
        fclose(pMembres);
        fopen_s(&pMembres, "membre.dat", "rb+");
    } while (struct_emprunt_ouvrages.id_membre != struct_acces_membre.id_membre);

    printf(COLOR_BOLD "Entrez le code ISBN a(10 chiffre) du livre :\n>" COLOR_OFF);
    gets_s(struct_emprunt_ouvrages.isbn, TAILLE_ISBN_OUVRAGE);
    while (fread_s(&struct_emprunt_ouvrages_bd, sizeof(struct_emprunt_ouvrages_bd),
sizeof(struct_emprunt_ouvrages_bd), 1, pEmprunt) != 0 && decideDePayer)
    {
        ecart_date_emprunt = calculerNbJours(struct_emprunt_ouvrages_bd.date_emprunt,
date_auj);
        if (strcmp(struct_emprunt_ouvrages.isbn, struct_emprunt_ouvrages_bd.isbn) == 0)
        {
            if (ecart_date_emprunt > 14)
            {
                nb_calcul_amende = ecart_date_emprunt - 14;
                montant_amende = nb_calcul_amende * 0.10;
                red();
                printf("vous devez payer une amende de %.2f euros\n",
montant_amende);
                reset();
                printf(COLOR_BOLD "voulez-vous payer l'amende maintenant?:\n
n>" COLOR_OFF);
                printf(COLOR_BOLD "Entrez o = oui ou n = non:\n>"
COLOR_OFF);
                cyan();
                scanf_s("%c", &validation_payer_amende, 1);
                viderTampon();
                reset();
                if (validation_payer_amende == 'o')
                {
                    montant_amende = 0;

```

```

                                struct_emprunt_ouvrages_bd.date_emprunt = date_auj;
                                fseek(pEmprunt, -1 *
(long)sizeof(struct_emprunt_ouvrages_bd), SEEK_CUR);
                                fwrite(&struct_emprunt_ouvrages_bd,
sizeof(struct_emprunt_ouvrages_bd), 1, pEmprunt);
                                }
                                else
                                {
                                    decideDePayer = false;
                                }

                                }
                                else
                                {
                                    green();
                                    printf("vous n'avez pas d'amende a payer\n");
                                    reset();
                                }

                                }

                                }
                                fclose(pMembres);
                                fclose(pEmprunt);
}

```

3.1. Structures et énumérations

```
struct Ouvrage {
    char isbn[TAILLE_ISBN_OUVRAGE];
    char titre[TAILLE_TITRE_OUVRAGE];
    char auteur[TAILLE_AUTEUR_OUVRAGE];
    int annee_parution;
    char editeur[TAILLE_EDITEUR_OUVRAGE];
};
typedef struct Ouvrage ouvrage;
struct Membre {
    int id_membre;
    char prenom[TAILLE_PRENOM_MEMBRE];
    char nom[TAILLE_NOM_MEMBRE];
    char adresse[TAILLE_ADRESSE_MEMBRE];
    int date_adhesion;
};
typedef struct Membre membre;
struct Emprunt {
    int id_membre;
    char isbn[TAILLE_ISBN_OUVRAGE];
    int date_emprunt;
};
typedef struct Emprunt emprunt;
```

3.2. Prototypes

Fonction « ouverture_Fichiers »

```
void ouverture_Fichiers(void);
```

Entrée rien

Sortie : rien

Fonction « affichage_Menu_et_demanderChoix »

```
int affichage_Menu_et_demanderChoix(void);
```

Entrée rien

Sortie : le nombre choisis par l'utilisateur

Fonction « gestion_livres »

```
void gestion_livres(void);
```

Entrée rien

Sortie : rien

Fonction «gestion_membres»

`void gestion_membres(void);`

Entrée rien

Sortie : rien

Fonction «gestion_emprunts_et_remises_des_livres»

`void gestion_emprunts_et_remises_des_livres(void);`

Entrée rien

Sortie : rien

Fonction «ajouter_nouveau_livre»

`void ajouter_nouveau_livre(void);`

Entrée rien

Sortie : rien

Fonction «modifier_livre»

`void modifier_livre(void);`

Entrée rien

Sortie : rien

Fonction «lister_livre»

`void lister_livre(void);`

Entrée rien

Sortie : rien

Fonction «recherche_livre»

`void recherche_livre(void);`

Entrée rien

Sortie : rien

Fonction «supprimer_livre»

Void supprimer_livre(**void**);

Entrée rien

Sortie : rien

Fonction «enregistrer_new_membre»

void enregistrer_new_membre(**void**);

Entrée rien

Sortie : rien

Fonction «modifier_donnee_membre»

void modifier_donnee_membre(**void**);

Entrée rien

Sortie : rien

Fonction «supprimer_membre»

void supprimer_membre(**void**);

Entrée rien

Sortie : rien

Fonction «affichage_gestion_livres_demande_choix»

int affichage_gestion_livres_demande_choix(**void**);

Entrée rien

Sortie : le nombre choisis par l'utilisateur

Fonction «affichage_gestion_membres_demande_choix»

int affichage_gestion_membres_demande_choix(**void**);

Entrée rien

Sortie : le nombre choisis par l'utilisateur

Fonction «affichage_gestion_emprunts_remises_demande_choix»

int affichage_gestion_emprunts_remises_demande_choix(**void**);

Entrée rien

Sortie : le nombre choisis par l'utilisateur

Fonction «emprunts_ouvrages»

`void emprunts_ouvrages(void);`

Entrée rien

Sortie : rien

Fonction «remises_ouvrages»

`void remises_ouvrages(void);`

Entrée rien

Sortie : rien

Fonction «payer_amende»

`void payer_amende(void);`

Entrée rien

Sortie : rien

Fonction «afficher_liste_membres»

`void afficher_liste_membres(void);`

Entrée rien

Sortie : rien

Fonction «viderTampon»

`void viderTampon(void);`

Entrée rien

Sortie : rien

Fonction «obtenir_annee»

```
int obtenir_annee(void);
```

Entrée rien

Sortie : l'année

3.3. Fonctions

Décrivez l'ensemble des fonctions de votre programme (fonction principale et fonctions auxiliaires). L'intérêt ici est d'expliquer plus en détail votre implémentation.

4. Rétrospective.

Forces et Faiblesse :

- ses forces sont que toutes les fonctions fonctionnent mais le problème c'est qu'il y a des erreurs.
- toutes les fonctions voulues ont pu être implémenter.
- ses faiblesses sont qu'il y a des erreurs dans mon code.

5. Perspectives

On peut améliorer que pour les remises et pour emprunter un livre il manque une belle recherche et une boucle pour quand l'on se trompe d'ISBN ou d'ID cela nous le redemande tant que l'ISBN ou l'ID n'est pas correct.

On peut imaginer une liste des emprunts ou des amendes.

6. Annexe : rencontre avec les différents acteurs

« Nous sommes une petite communauté bénévoles qui met à disposition des livres à un prix attractif. Nous avons regroupé un ensemble d'ouvrages venant de quelques déstockages. Nous les avons tous listés avec leur référence (il s'agit d'un code ISBN à 10 chiffres), l'auteur, l'éditeur, l'année de parution, le titre. La bibliothèque ne possède pas beaucoup de place, ainsi nous avons comme parti pris que nous ne posséderons jamais deux fois le même livre.

Avant que la bibliothèque ne soit ouverte, la première chose que nous souhaiterions c'est de mettre en place un système qui nous permette de lister tous ces ouvrages, de pouvoir en ajouter de nouveaux, de pouvoir modifier le contenu d'un ouvrage ou de pouvoir en supprimer. »

Peter : Président

« Nous souhaiterions également pouvoir indiquer leur statut. S'ils sont libres où empruntés. Bien entendu, chaque personne qui emprunterait un livre serait obligatoirement enregistrée dans le système. Ils seraient membres en quelque sorte. Nous conserverions leur nom, prénom, adresse, ainsi qu'un numéro d'identification et leur date d'adhésion à la bibliothèque. L'enregistrement est gratuit. . Tout comme les livres nous pourrions modifier les données d'un membre, ou le supprimer ou en ajouter un nouveau. »

Auguste : gestionnaire des stocks

« Les emprunts fonctionnent de la manière suivante. Pour chaque livre, l'emprunt coûte 0,75€. L'emprunt est fixé à deux semaines. Un membre ne peut emprunter plus de 10 livres à la fois. Une amende de 0,10€ par jour est notée pour chaque livre pour chaque semaine de retard. Si un membre possède une amende impayée et/ou un livre en retard, il ne peut plus emprunter de nouveau livre.

J'imagine un écran pour l'emprunt où l'on insère le numéro du membre et les différents livres qu'il emprunte. S'affiche alors le prix à payer. Nous validerions nous même le paiement puisque nous n'accepterons que l'argent liquide.

Il serait intéressant d'avoir une partie dans le programme destinée à la remise des livres. On insère leur ISBN et on sait qu'ils sont à nouveau disponibles. S'il y a une amende à payer, elle est indiquée à l'écran. Nous validerions ensuite si l'utilisateur a bien payé son amende. »

Ce qui est également très sollicité, c'est la recherche. Ne fusse que sur un code ISBN. Pour que l'on puisse déterminer si nous possédons l'ouvrage, s'il est disponible ou sinon à quelle date il est supposé revenir. »

Rosie : accueil

« La priorité du logiciel doit être la gestion du stock. Puisque nous gardons une comptabilité papier, la gestion des prix et des amendes peut être intégrée dans un second temps. »

Peter : Président

«Je pense pouvoir vous fournir le fichier complet avec le contenu complet des références. Pensez-vous qu'il soit possible que votre logiciel mette à jour ce fichier texte ? Ainsi nous pourrions également toujours pouvoir ressortir la liste des ouvrages que nous possédons. Il faudrait en fait que le logiciel conserve également un fichier pour les membres. Et je pense très probablement pour les emprunts en cours. »

David: Statistiques